

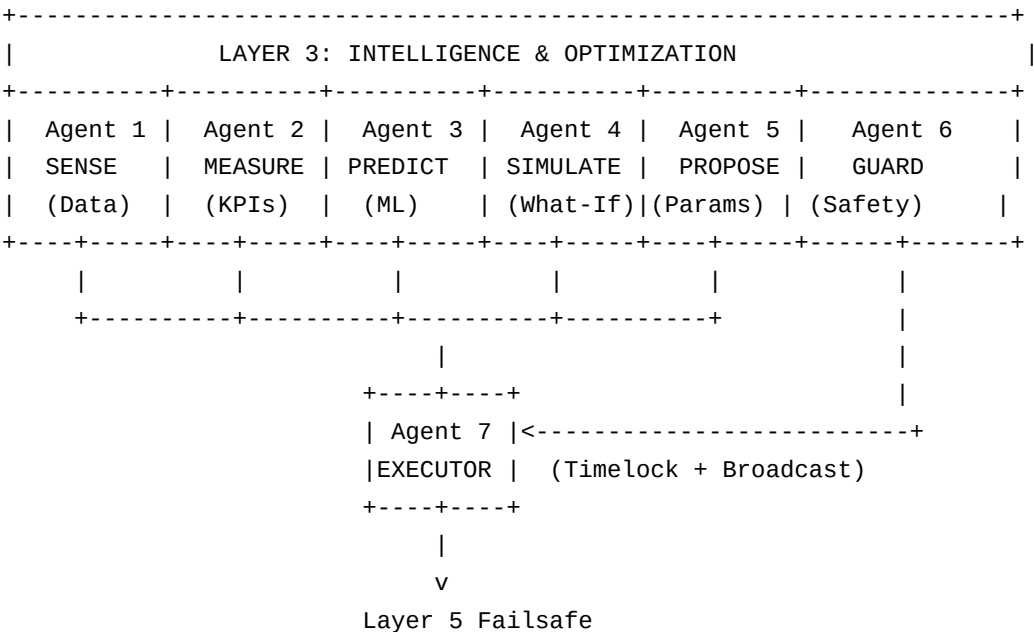
Horizontal Decomposition of Layer 3

Intelligence & Optimization Layer

Agentic Coding Architecture

Layer 3 is decomposed into 7 horizontally stacked, autonomous agents that communicate via immutable event buses and shared state contracts. Each agent is independently deployable, testable, and replaceable. No agent has write access to another's internal state.

Architecture Overview



Layer 3: Intelligence & Optimization -- Horizontal Decomposition

Agent 1: Data Ingestion & Feature Engineering Agent (sense-agent)

Responsibility: Consume raw data from Layer 4 oracles and Layer 2 state, transform into ML-ready feature vectors.

Input Contracts

```
{
  "on_chain_metrics": ["voter_turnout", "quadratic_distribution", "treasury_velocity",
"proposal_count", "pass_rate"],
  "off_chain_signals": ["token_price", "sentiment_score", "developer_activity"],
  "identity_layer_events": ["new_verifications", "sybil_flags", "credit_allocations"]
}
```

Output Contracts

```
{
  "feature_vector": {
    "window_start": "timestamp",
    "window_end": "timestamp",
    "governance_health_features": {},
    "voter_behavior_features": {},
    "economic_context_features": {}
  },
  "data_provenance_hash": "sha256",
  "oracle_attestations": ["chainlink_proof_1", "chainlink_proof_2"]
}
```

Agentic Coding Notes

- Stateless transformation pipeline. Each run is deterministic given the same inputs.
- Must validate cryptographic proofs before processing (fails fast).
- Emit FeatureReady event; never write to Layer 2 directly.

Layer 3: Intelligence & Optimization -- Horizontal Decomposition

Agent 2: KPI Calculation & Health Monitoring Agent (measure-agent)

Responsibility: Compute governance KPIs from feature vectors, detect anomalies, trigger optimization cycles.

KPI Registry (Hardcoded Formulas)

KPI	Formula	Target Range
voter_apathy_index	$1 - (\text{actual_votes} / \text{eligible_voters})$	< 0.6
quadratic_gini	Gini coefficient of credit expenditure	< 0.4
proposal_success_rate	$\text{passed_proposals} / \text{total_proposals}$	0.3 - 0.7
treasury_efficiency	$\text{executed_value} / \text{proposed_value}$	> 0.5
sybil_risk_score	Derived from identity layer flags	< 0.1

Output Contracts

```
{
  "kpi_snapshot": {
    "timestamp": "iso8601",
    "metrics": { "voter_apathy_index": 0.72, "quadratic_gini": 0.35 },
    "health_status": "DEGRADED" | "HEALTHY" | "CRITICAL",
    "anomaly_flags": ["voter_apathy_index_exceeded"]
  }
}
```

Agentic Coding Notes

- Pure function: `calculate_kpis(feature_vector) -> kpi_snapshot`
- Emits `OptimizationTrigger` only when KPIs breach thresholds or on scheduled cadence.
- No ML here -- deterministic, auditable math only.

Layer 3: Intelligence & Optimization -- Horizontal Decomposition

Agent 3: Optimization Engine Agent (predict-agent)

Responsibility: The "Brain." Run ML/optimization algorithms to suggest parameter adjustments.

Optimization Variables (Bounded)

```
OPTIMIZATION_SPACE = {  
    "quorum_threshold": {"min": 0.04, "max": 0.20, "current": 0.10},  
    "voting_duration_blocks": {"min": 1000, "max": 10000, "current": 5000},  
    "voice_credit_regen_rate": {"min": 1.0, "max": 5.0, "current": 2.0},  
    "proposal_bond": {"min": 100, "max": 10000, "current": 1000}  
}
```

Algorithm Modules (Pluggable)

- BayesianOptimizer: For continuous parameter tuning
- BanditAlgorithm: For exploration/exploitation of voting mechanisms
- NeuralSurrogate: Trained on historical (features, outcomes) pairs

Output Contracts

```
{  
    "optimization_result": {  
        "recommended_params": {  
            "quorum_threshold": 0.08,  
            "voting_duration_blocks": 4500,  
            "voice_credit_regen_rate": 2.5  
        },  
        "confidence_score": 0.87,  
        "expected_improvement": {  
            "voter_apathy_index": -0.15,  
            "proposal_success_rate": +0.10  
        },  
        "model_version": "v2.3.1",  
        "training_data_hash": "sha256"  
    }  
}
```

Agentic Coding Notes

- Runs off-chain or in verifiable coprocessor (TEE/ZK-ML).
- Must version models and expose training provenance.
- Never outputs values outside hardcoded bounds -- enforced at code level, not just policy.

Layer 3: Intelligence & Optimization -- Horizontal Decomposition

Agent 4: Simulation & Backtesting Agent (simulate-agent)

Responsibility: Validate optimization recommendations against historical data before they become proposals.

Mechanism

1. Take recommended_params from Agent 3
2. Replay last N governance cycles with counterfactual parameters
3. Calculate simulated KPIs
4. Flag if simulation predicts worse outcomes or edge-case failures

Output Contracts

```
{
  "simulation_report": {
    "backtest_period": {"start": "t-90d", "end": "t"},
    "historical_scenarios_run": 150,
    "simulated_kpis": { "voter_apathy_index": 0.58, "proposal_success_rate": 0.45 },
    "vs_current_kpis": { "voter_apathy_index": -0.14, "proposal_success_rate": +0.08 },
    "edge_case_flags": ["low_turnout_weekend_effect"],
    "recommendation": "PASS" | "FAIL" | "PASS_WITH_CAVEATS"
  }
}
```

Agentic Coding Notes

- Acts as a circuit breaker for bad ML predictions.
- Must be deterministic (fixed random seeds, reproducible environments).
- Emits SimulationPassed or SimulationFailed event.

Layer 3: Intelligence & Optimization -- Horizontal Decomposition

Agent 5: Parameter Proposal Generator Agent (propose-agent)

Responsibility: Convert validated optimization results into Layer 2-compatible parameter update transactions.

Function

- Encode recommended_params into a ParameterAdjustment struct
- Generate the Layer 2 transaction payload (but do NOT sign or submit)
- Attach all upstream provenance (Agent 1->2->3->4 outputs)

Output Contracts

```
{
  "parameter_proposal": {
    "target_contract": "Layer2_Governance_Engine",
    "function_signature": "adjustParameters(ParameterAdjustment)",
    "call_data": "0x...",
    "proposed_values": { ... },
    "justification_bundle": {
      "kpi_snapshot_hash": "0x...",
      "optimization_result_hash": "0x...",
      "simulation_report_hash": "0x..."
    },
    "effective_epoch": 1523
  }
}
```

Agentic Coding Notes

- Stateless formatter. No business logic -- just data marshalling.
- The justification bundle is critical for Layer 5 human veto review.

Layer 3: Intelligence & Optimization -- Horizontal Decomposition

Agent 6: Safety Guardrails & Constraint Validation Agent (guard-agent)

Responsibility: The final mathematical safety check before any parameter adjustment reaches Layer 2.

Validation Rules (Hardcoded, Immutable)

1. Bounds Check: Every proposed value within [hardcoded_min, hardcoded_max]
2. Rate Limiting: $|\text{new_value} - \text{old_value}| / \text{old_value} < \text{max_delta_per_epoch}$ (e.g., 20%)
3. Treasury Isolation: Verify call_data does NOT contain treasury transfer functions
4. No Force-Execute: Verify call_data does NOT contain proposal execution functions
5. Rollback Safety: Verify the adjustment has a corresponding safe_baseline revert target

Output Contracts

```
{
  "safety_verdict": {
    "status": "APPROVED" | "REJECTED",
    "rejection_reason": "BOUNDS_VIOLATION" | "RATE_LIMIT_EXCEEDED" | "TREASURY_TOUCH",
    "checksums": {
      "bounds_check": "PASSED",
      "rate_limit_check": "PASSED",
      "treasury_isolation": "PASSED"
    }
  }
}
```

Agentic Coding Notes

- This is the non-negotiable gate. Code it defensively; assume upstream agents are compromised.
- If REJECTED, emit SafetyHalt event and notify Layer 5.
- Should be minimal, formally verifiable logic (consider Coq/TLA+ specs).

Layer 3: Intelligence & Optimization -- Horizontal Decomposition

Agent 7: Execution Orchestrator Agent (executor-agent)

Responsibility: Coordinate the end-to-end flow and interface with Layer 5 (Failsafe/Timelock).

State Machine

IDLE -> DATA_COLLECTED -> KPIS_CALCULATED -> OPTIMIZED -> SIMULATED
-> PROPOSED -> SAFETY_PASSED -> TIMELOCK_QUEUED -> AWAITING_VETO -> EXECUTED

Functions

- Subscribe to events from Agents 1-6
- Maintain workflow state in a persistent event log
- Upon SafetyPassed, submit proposal to Layer 5 Timelock contract
- Monitor veto window; if clear, broadcast execution to Layer 2
- If veto triggered, initiate safe_baseline rollback

Output Contracts

```
{
  "orchestration_log": {
    "cycle_id": "uuid",
    "status": "TIMELOCK_QUEUED",
    "layer5_tx_hash": "0x...",
    "veto_deadline": "timestamp",
    "rollback_ready": true
  }
}
```

Agentic Coding Notes

- The only agent with external network permissions (Layer 5 / Layer 2).
- Must be idempotent -- safe to restart without double-submitting.
- Maintain a safe_baseline config snapshot for instant rollback.

Cross-Cutting Concerns (Agentic Infrastructure)

1. Event Bus Schema

```
message Layer3Event {  
    string agent_id = 1;  
    string event_type = 2;  
    bytes payload_hash = 3;  
    bytes payload = 4;  
    uint64 timestamp = 5;  
    bytes signature = 6; // Agent's identity key  
}
```

2. Agent Isolation Boundaries

Concern	Rule
State	No shared memory. Agents read from event bus only.
Secrets	Agent 3 may hold model weights; no other agent accesses them.
Network	Only Agent 7 has outbound blockchain RPC access.
Upgrade	Agents 1-6 are independently versioned. Agent 7 is minimal and rarely changes.

3. Failure Modes

Failure	Response
Agent 1 (Data) stalls	Agent 2 emits DataStale -> skip cycle
Agent 3 (ML) outputs garbage	Agent 4 catches in simulation -> reject
Agent 6 (Guard) rejects	Halt cycle, notify Layer 5, preserve logs
Agent 7 (Executor) crashes	On restart, replay event log to recover state

Layer 3: Intelligence & Optimization -- Horizontal Decomposition

Implementation Roadmap for Agentic Coding

Phase A: Skeleton (Week 1)

Implement Agent 2 (KPIs) and Agent 6 (Guard) first -- these are deterministic and define the system's safety envelope. Wire them with a mock event bus.

Phase B: Data Pipeline (Week 2)

Implement Agent 1 (Data) with real Layer 4 oracle adapters. Implement Agent 5 (Proposal Generator) as a passthrough formatter.

Phase C: Intelligence (Week 3)

Implement Agent 3 with a simple Bayesian optimizer or even a rules-based heuristic initially. Implement Agent 4 with a basic backtester (replay last 30 days).

Phase D: Orchestration (Week 4)

Implement Agent 7 with full Layer 5 timelock integration. End-to-end integration test with Layer 2 testnet.

This decomposition ensures each agent is small enough to audit, large enough to be useful, and isolated enough to fail safely.